

文章笔记

用 Karpathy 的 AutoResearch 方法将 Claude Skills 效果提升 10 倍

五道口纳什 & Claude

2026 年 3 月 29 日

原文作者: Ole Lehmann (@itssolelehmann)

原文标题: How to 10x your Claude Skills (using Karpathy's autoresearch method)

原文链接: x.com/itssolelehmann/status/2033919415771713715

发布日期: 2026 年 3 月 17 日

译者参考: 实践哥 MinLi (@MinLiBuilds)

关于本文

本笔记基于 Ole Lehmann 的英文原文整理。Ole Lehmann 是 AI 领域极具影响力的内容创作者，专注于 AI 商业自动化和个人生产力，拥有超过 3.4 万订阅者。文章的核心价值在于：将 Karpathy 提出的 autoresearch 方法应用于 Claude Skills 的自动化质量提升，任何可度量的 skill 都可以通过这套“评估-优化-评估”的循环持续改进。

目录

1 问题：你的 Skill 可能有 30% 的时间在悄悄失效	2
1.1 本章小结	2
2 AutoResearch 方法的起源	2
2.1 Karpathy 的原始方法	2
2.2 作者的改造	2
2.3 本章小结	3
3 一次循环的工作原理	3
3.1 菜谱类比	3
3.2 映射到 Skill 优化	3
3.3 本章小结	3
4 Checklist：告诉 Agent “什么叫好”	4
4.1 你在整个过程中唯一的工作	4
4.2 为什么用是/否而不用打分	4
4.3 落地页文案 Checklist 示例	4
4.4 Checklist 的生成与数量	4
4.5 本章小结	5
5 实操步骤：如何运行 AutoResearch	5
5.1 本章小结	6
6 实战案例：落地页文案 Skill 优化	6
6.1 优化结果	6
6.2 Agent 具体做了哪些改动	6
6.3 最终产出物	6
6.4 本章小结	7
7 更广泛的应用场景	7
7.1 具体案例	7
7.2 本章小结	7
8 总结与延伸	8
8.1 核心要点回顾	8
8.2 方法论启示	8
8.3 延伸阅读	8

1 问题：你的 Skill 可能有 30% 的时间在悄悄失效

你的 Claude skill 可能有 30% 的时间输出质量不达标，而你根本没有意识到。

作者开篇就抛出了一个尖锐的判断：大多数人搭建了 AI skill 后就“用着”，从不系统性地检验输出质量。他本人也经历过这个阶段——直到他搭了一套方法，能在 autopilot 模式下自动迭代任何 skill。

核心数据

作者的落地页文案 skill 质量检查通过率从 **56% 提升到 92%**，全程零手动干预。Agent 自行不断测试和收紧 prompt，完全自主运行。

这套方法的思路是：你启动它，agent 就反复测试和改进 skill，不需要你动手。下面将完整拆解这套方法以及如何在你自己的 skill 上运行它。

1.1 本章小结

Skill 的失效往往是隐蔽的，大多数人缺乏系统化的评估手段。AutoResearch 方法提供了一种全自动的质量迭代方案，能在无人干预的情况下持续提升 skill 的输出质量。

2 AutoResearch 方法的起源

2.1 Karpathy 的原始方法

Andrej Karpathy (OpenAI 联合创始人、特斯拉前 AI 负责人、“vibe coding”一词的发明者) 发布了一套叫做 **autoresearch** 的方法。

AutoResearch 核心思路

不让你手动改进，而是让 AI agent 在循环里替你做：

1. 尝试一个小改动
2. 检查结果是否变好
3. 变好了就留下，没变好就扔掉
4. 然后再来一次，再来一次……

Karpathy 最初把这套方法用在机器学习代码上。但关键洞察在于：这套方法适用于**任何你能衡量并改进的东西**——包括你在 Claude 里搭建的 skills。

2.2 作者的改造

作者将 Karpathy 的方法改造成了一个可以在 Claude Code 和 Cowork 里运行的 skill。使用方式极其简单：

“run autoresearch on my landing page skill”

说一句话，agent 就会接管整个优化过程。

2.3 本章小结

AutoResearch 源自 Karpathy，核心是“小改动 → 评估 → 保留或撤销”的自动循环。作者将其从机器学习场景推广到 Claude Skills 优化，封装成可直接调用的 skill。

3 一次循环的工作原理

3.1 菜谱类比

作者用一个直观的类比来解释 autoresearch 的单次循环：

菜谱类比——理解 AutoResearch 循环

你有一个菜谱，十次里有七次做得不错，另外三次总是不对劲——酱汁太淡，或者调味不对。你不是从头重写整个菜谱，而是：

1. 换一种配料
2. 用这个改动做十次
3. 变好了？留下这个改动
4. 变差了？换回原来的配料
5. 改下一个东西，再做十次

经过 50 轮这样的迭代，你的菜谱十次里有 9.5 次都能成功。

3.2 映射到 Skill 优化

这个类比精确对应到 skill 优化的每个环节：

- 菜谱 (recipe) = 你的 skill prompt
- 做菜 (cooking) = 运行这个 skill
- 试味 (tasting) = 给输出结果打分

而你唯一需要提供的，就是**打分标准** (scoring criteria)。

3.3 本章小结

单次循环的逻辑是：改一个点 → 测试多次 → 根据分数决定保留或撤销。通过大量循环的累积，skill 的整体质量从”七成靠谱”逐步提升到”九成五靠谱”。这是一个经典的贪心搜索过程。

4 Checklist：告诉 Agent “什么叫好”

4.1 你在整个过程中唯一的工作

你唯一要做的事

给 agent 一份简单的 checklist，定义“什么叫好”。这是你在整个 autoresearch 过程中唯一需要参与的环节。

Checklist 由简单的是/否问题组成，每个问题检查输出的一个具体方面。通过或失败，就这么简单。Agent 用这份 checklist 对每次输出打分，分数变化告诉它改动是在帮忙还是在帮倒忙。

4.2 为什么用是/否而不用打分

作者用“老师批卷子”来说明为什么要用二元判断而非主观评分。

不是“给写作质量打个 1-10 分”（模糊、每次结果不一样），而是每一项都清清楚楚是或否：

- 学生有没有写论点陈述（thesis statement）？是或否。
- 每处引用都注明出处了吗？是或否。
- 篇幅在 5 页以内吗？是或否。

用这份 checklist 批 100 份卷子，每次结果都一致。

一致性是关键

主观评分（如 1-10 分）在不同时间、不同上下文下会产生不同结果，导致评估信号噪声大。是/否二元判断消除了这种不确定性，让 agent 能可靠地判断改动的效果方向。

4.3 落地页文案 Checklist 示例

以下是作者给出的落地页文案 skill 的具体 checklist 问题：

1. “标题有没有包含具体数字或结果？”——捕捉“Grow Your Business”这类模糊标题。
2. “文案中有没有 revolutionary、synergy、cutting-edge、next-level 这类流行词？”——捕捉空洞的 buzzword 堆砌。
3. “CTA 是否使用了具体的动词短语？”——捕捉“Learn More”或“Click Here”这类弱 CTA。
4. “第一句话有没有点出一个具体的痛点？”——捕捉“In today’s fast-paced world...”这类泛泛的开场。
5. “总文案字数是否在 150 词以内？”——捕捉臃肿的页面。

4.4 Checklist 的生成与数量

这些 checklist 不需要你自己费劲想。启动 autoresearch 时，agent 会引导你完成整个过程：

- 问你“什么叫好”

- 帮你把模糊的感觉 (vibes) 变成具体的是/否问题
- 如果你有现成的 style guide, 还能从中提取标准

Checklist 数量的陷阱

3–6 个问题是最佳数量。超过这个范围, skill 会开始“应付 checklist”——就像学生背答案而不理解题目。这是一种 overfitting 现象: skill 在 checklist 上得分越来越高, 但实际输出质量反而下降。

4.5 本章小结

评分标准是 autoresearch 中唯一需要人类定义的部分。使用是/否二元判断 (而非主观评分) 确保评估的一致性和可复现性。Checklist 应控制在 3–6 个问题, 避免 overfitting。Agent 会引导你生成 checklist, 你不需要从零设计。

5 实操步骤: 如何运行 AutoResearch

以下是运行 autoresearch 的完整 6 步流程:

1. **Step 1: 下载 skill。**获取 autoresearch skill, 放进 Claude Code 或 Cowork 的 skills 文件夹。
2. **Step 2: 选一个要改进的 skill。**说 “run autoresearch on my [skill name] skill”。选最让你头疼的那个——时好时坏、输出不稳定的 skill。
3. **Step 3: Agent 问你 3 件事:**
 - 要优化哪个 skill
 - 用什么测试输入 (例如 “write landing page copy for an AI productivity tool”)
 - 你的 checklist 问题是什么
4. **Step 4: 跑一遍 skill, 给出起始分数。**这是基准线 (baseline)。作者的落地页 skill 起步只有 56%——标题模糊、buzzword 泛滥、CTA 软弱, 超过一半的检查项都没通过。
5. **Step 5: 浏览器弹出实时 dashboard。**包含以下内容:
 - 分数随时间变化的曲线图
 - 每项 checklist 的通过/失败明细
 - 每次改动的日志
 - 每 10 秒自动刷新
6. **Step 6: 走开。**Agent 进入循环:
 - 分析哪些检查项在失败
 - 对 skill prompt 做一个小改动
 - 重新测试
 - 分数升就留下改动, 分数降就撤销

- 一直自主运行，直到你叫停，或连续三次达到 95%+ 为止

全自动运行，原始 Skill 不受影响

你可以盯着 dashboard 看，也可以完全走开。Agent 不需要你参与任何决策。改进后的版本保存为单独的文件，你的**原始 skill 始终不会被修改**。

5.1 本章小结

实操分为 6 步：下载 skill → 选目标 → 回答 3 个问题 → 建立基准线 → 观察 dashboard → 放手让 agent 跑。整个过程除了初始配置外完全自动化，且原始 skill 始终安全。

6 实战案例：落地页文案 Skill 优化

6.1 优化结果

作者在自己的落地页文案 skill 上运行 autoresearch，结果如下：

通过率：56% → 92%

共 4 轮改动 (rounds of changes)，其中 3 个保留，1 个撤销。

6.2 Agent 具体做了哪些改动

1. **针对最高频失败项加了明确规则**：“Your headline must include a specific number or result. Never use vague promises like ‘Transform Your Business.’” ——直接堵住最常见的失败模式。
2. **加了禁用词列表 (banned buzzwords list)**：明确列出禁止使用的词汇——revolutionary, cutting-edge, synergy, next-level, game-changing, leverage, unlock, transform。
3. **加了一个 worked example**：在 skill prompt 中加入一段实际的高质量落地页示例，标注了痛点开场白 (pain point opener) 和 CTA 的位置，让 skill 能**直接看到“好”的样子**，而不是靠猜。
4. **尝试更严格的字数限制 (被撤销)**：文案变得太单薄 (too thin)，CTA 质量也下降了。系统检测到了这种“单独看似改进、但实际损害整体输出”的改动，自动撤销。

改动被撤销的智慧

第 4 轮改动展示了 autoresearch 的一个关键能力：它不是盲目叠加约束，而是能识别**局部改进但全局退步**的情况。更严的字数限制让“字数在 150 词以内”这项检查更容易通过，但牺牲了 CTA 质量和内容丰富度。系统权衡了整体得分后选择撤销。

6.3 最终产出物

运行结束后，你会得到以下 4 样东西：

1. **改进后的 skill**——保存为单独文件，原版完好无损，随时可回退。
2. **结果日志 (results log)** ——记录每一轮的得分。

3. **Changelog**——解释每次改动的内容、agent 为什么尝试这个改动、以及改动是否有效。
4. **原始 skill 的备份**——以防你需要回到起点。

Changelog 是最有价值的产出

那份 changelog 可能是整个过程中最值钱的东西。它是一份完整的记录：对于这个特定的 skill，什么有效、什么无效。
更重要的是，当更强的模型问世时，你把这份 changelog 交给新模型，它就能从上一个 agent 停下的地方**接着优化**。Changelog 让 skill 的改进经验变成了可传承的资产。

6.4 本章小结

实战结果证明了 autoresearch 的有效性：4 轮自动迭代将通过率从 56% 提升到 92%。系统能自动识别有害改动并撤销。最终产出不仅是优化后的 skill，还包括完整的 changelog，为后续持续优化和跨模型传承提供基础。

7 更广泛的应用场景

AutoResearch 的适用范围远不止 Claude skill。作者明确指出：**任何你能打分的东西，都能用这套方法。**

7.1 具体案例

- **网站速度优化**：改一处代码 → 测量页面加载时间 → 留下或撤销。有人在 67 轮迭代后将页面加载时间从 1100ms 降到 67ms。
- **Cold outreach (陌生客户开发邮件)**：定义 checklist——“有没有提到对方公司？是否在 75 字以内？是否以具体问题收尾？”让 agent 跑 50 个变体。
- **Newsletter 开篇**：“开场白有没有包含个人细节？”“有没有使用陈词滥调？”让 agent 在 autopilot 上打磨你的文字。
- **任何你反复使用的 prompt**——只要能定义打分标准，就能跑 autoresearch。

通用原则

If you can score it, you can autoresearch it.

关键前提只有一个：你的优化目标必须是可量化的。能用是/否 checklist 打分，就能用 autoresearch 自动迭代。

7.2 本章小结

AutoResearch 方法的唯一前提是“可量化”。从网站性能到邮件写作、从 newsletter 到任何反复使用的 prompt，只要能定义出是/否评分标准，就能套用这套自动迭代框架。

8 总结与延伸

8.1 核心要点回顾

1. **问题**：大多数人不知道自己的 AI skill 有 30% 的时间在输出低质量结果，缺乏系统化测试手段。
2. **方法来源**：借鉴 Karpathy 的 autoresearch，核心是“小改动 → 评估 → 保留或撤销”的自动循环。
3. **关键设计**：用 3-6 个是/否 checklist 问题定义“什么叫好”，确保评估一致、可复现。过多会导致 overfitting。
4. **实操流程**：6 步——下载 skill、选目标、回答 3 个问题、建立基准线、观察 dashboard、放手让 agent 跑。
5. **实证结果**：落地页文案 skill 从 56% 提升到 92%，4 轮迭代，全程零手动。
6. **普适性**：任何可量化的优化目标都适用——网站速度、cold outreach、newsletter、任何反复使用的 prompt。

8.2 方法论启示

从靠运气到靠系统

这篇文章最深刻的洞察不在于技术细节，而在于一个认知转变：

- “搭好就用”是第一阶段——你有了工具。
- “搭好、测过、持续迭代”才是成熟阶段——你有了**系统**。
- Changelog 的积累让 skill 的改进成为**可传承的资产**，而非一次性的调参。当更强的模型出来，changelog 就是你交给新模型的“经验包”。

8.3 延伸阅读

- 原文作者 Ole Lehmann 的 X 账号：@itsolelehmann
- 原文链接：How to 10x your Claude Skills
- Andrej Karpathy 的 autoresearch 原始方法与仓库
- AutoResearch skill 下载：见原文末尾 Dropbox 链接或作者 GitHub
- 译者实践哥 MinLi 的 X 账号：@MinLiBuilds